Encapsulation & Abstraction in C

Simulating Private Properties, Protected Inheritance, and Pure Virtual Functions

Topic Overview & Simulation Techniques

Unlike object-oriented languages (Java, C++, Python) which have built-in keywords like `private`, `protected`, and `abstract`, the C programming language achieves these exact architectural paradigms through deliberate design conventions and language features:

- **Encapsulation (Private Properties):** Achieved using opaque structs (forward declarations where the full definition is hidden in the implementation file) or static scoping, forcing access exclusively through getters and setters.
- **Protected Access:** Implemented via struct embedding (composition) combined with documentation conventions, allowing subclass functions to access parent members directly while keeping them hidden from general client code.
- **Abstraction (Pure Virtual Functions):** Modeled using structs containing function pointers. Base pointers can define default abstract error behaviors that subclasses override, mimicking runtime polymorphism and interface contracts.

Task Solutions

Task 1: Class Song with Private Properties & Getters/Setters

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* Forward declaration: struct internals are hidden from users */
typedef struct Song Song;

struct Song {
    char title[100];   /* private */
    char artist[100];  /* private */
};

Song* Song_create(const char *title, const char *artist) {
    Song *s = malloc(sizeof(Song));
    strncpy(s->title, title, sizeof(s->title) - 1);
    strncpy(s->artist, artist, sizeof(s->artist) - 1);
    return s;
}

const char* Song_getTitle(Song *s) { return s->title; }
void Song_setTitle(Song *s, const char *title) {
    strncpy(s->title, title, sizeof(s->title) - 1);
}

const char* Song_getArtist(Song *s) { return s->artist; }
void Song_setArtist(Song *s, const char *artist) {
```

```
    strncpy(s->artist, artist, sizeof(s->artist) - 1);
}

void Song_destroy(Song *s) { free(s); }
```

Task 2: InstaStory with Protected Property & SponsoredStory

```
typedef struct {
    int storyViews;    /* PROTECTED: intended for subclasses */
} InstaStory;

void InstaStory_init(InstaStory *self, int views) {
    self->storyViews = views;
}

typedef struct {
    InstaStory base;   /* Inherited parent part */
    char sponsorName[50];
} SponsoredStory;

void SponsoredStory_init(SponsoredStory *self, int views, const char *
    sponsor) {
    InstaStory_init(&self->base, views);
    strncpy(self->sponsorName, sponsor, sizeof(self->sponsorName) - 1);
}

void SponsoredStory_display(SponsoredStory *self) {
    printf("Sponsor     : %s\n", self->sponsorName);
    printf("Story Views : %d (accessed via protected member)\n", self->base
        .storyViews);
}
```

Task 3: Abstract Class Product with Abstract upload()

```
typedef struct Product Product;

struct Product {
    char name[50];
    void (*upload)(Product *self);  /* Pure virtual function pointer */
};

void Product_abstractUpload(Product *self) {
    printf("ERROR: upload() is abstract! %s must implement it.\n", self->
        name);
}

void Product_init(Product *self, const char *name) {
    strncpy(self->name, name, sizeof(self->name) - 1);
    self->upload = Product_abstractUpload;
}

typedef struct {
    Product base;
    int warrantyMonths;
```

```
} Electronics;

void Electronics_upload(Product *self) {
    Electronics *e = (Electronics*)self;
    printf("[%s] uploaded: Warranty (%d months). [OK]\n", e->base.name, e->
       warrantyMonths);
}
```

Task 4: Refactored `UserProfile` with Private Phone Number

```
typedef struct UserProfile UserProfile;

struct UserProfile {
    char name[50];
    char phoneNumber[15];  /* PRIVATE: reachable only via verified methods
       */
};

UserProfile* UserProfile_create(const char *name) {
    UserProfile *u = malloc(sizeof(UserProfile));
    strncpy(u->name, name, sizeof(u->name) - 1);
    strcpy(u->phoneNumber, "");
    return u;
}

void UserProfile_setPhoneNumber(UserProfile *u, const char *phone) {
    if (strlen(phone) != 10) {
        printf("Invalid phone number! Must be 10 digits.\n");
        return;
    }
    strncpy(u->phoneNumber, phone, sizeof(u->phoneNumber) - 1);
}
```

Encapsulation vs Abstraction Analysis

#	Concept	Core Definition & Real-World Analogy
1	**Encapsulation**	Hiding state and restricting direct access. *Example:* WhatsApp chat database is locked; access happens strictly via verified method APIs.
2	**Abstraction**	Hiding operational complexity behind simple interfaces. *Example:* Tapping the Instagram like button or share button without managing underlying server calls or compression queues.

Key Takeaway: Encapsulation safeguards data integrity through controlled getters and

setters, while abstraction simplifies system interaction by exposing only necessary behaviors through clean interfaces.